UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE

INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema: Ejercicios Guia APE
Unidad de Organización Curricular: BÁSICA
Nivel y Paralelo: 1 - B
Alumnos participantes: Leyber Peñafiel

Asignatura: Algoritmos y logica de programación
Docente: Ing. Jose Caizabuano, Mg.
Link de GitHub: https://github.com/LeyberP/EjerciciosGuiApe_Gp6

2.1 Objetivos

General: Resolver los ejercicios de la clase aplicando correctamente la declaración e invocación de métodos en Java, así como el uso de parámetros, argumentos y valores de retorno.

Específicos:

- Distinguir cuándo un método debe usar void y cuándo debe devolver un valor.
- Aplicar parámetros y argumentos en la definición y llamada de métodos.
- Implementar métodos de cálculo y validación, e interpretar el paso por valor de tipos primitivos.

2.2 Modalidad y tiempo de duración

Modalidad: práctica académica individual o colaborativa, según las indicaciones del docente.
Tiempo: no especificado en las diapositivas proporcionadas.

2.3 Instrucciones y recursos

Se desarrollan los ejercicios de la presentación de la Clase 3. Los ejemplos y soluciones están escritos en Java, el lenguaje utilizado en las diapositivas. Recurso: presentación de clase proporcionada por el estudiante.

2.4 Actividades por desarrollar y soluciones

Ejercicio en clase: completar la clase Producto

El método mostrarDetalles() solo imprime información, por eso usa void. El método calcularPrecioFinal() realiza un cálculo que debe entregarse al programa, por eso devuelve double y utiliza return.

```
public class Producto {
    String nombre;
    double precio;

    public void mostrarDetalles() {
        System.out.println("Producto: " + nombre + " | Precio: $" + precio);
    }

    public double calcularPrecioFinal(double porcentajeDescuento) {
        double descuento = precio * (porcentajeDescuento / 100);
        return precio - descuento;
    }
}
```

Ejercicio 1: identificación de tipos de retorno

La diapositiva enuncia el problema, pero no muestra el método que supuestamente produce el error. Por ello no es posible identificar el error exacto del fragmento original. La regla para corregirlo es: si el método declara un tipo distinto de void, debe devolver un valor compatible en todos los caminos de ejecución; si solo realiza una acción y no entrega un resultado, debe declararse void y no incluir un return con valor.

Ejemplo ilustrativo de un error y su corrección (no es una transcripción del código de la diapositiva):

```
// Incorrecto: declara double, pero no devuelve un valor
public double calcularArea(double lado) {
   System.out.println(lado * lado);
}

// Corrección: devuelve el resultado calculado
public double calcularArea(double lado) {
   return lado * lado;
}
```

Ejercicio 2: método void para una acción directa

Como el método únicamente imprime un mensaje y no necesita entregar un dato para otro cálculo, su tipo de retorno es void.

```
public class Bienvenida {
   public void saludar(String nombre, int edad) {
      System.out.println("Hola " + nombre + ", tienes " + edad
            + " años. ¡Bienvenido al sistema!");
   }
}
```

Ejercicio 3: parámetros vs. argumentos

El fragmento de código al que se refiere el enunciado no aparece en el contenido legible de la diapositiva. En el ejemplo ilustrativo siguiente, los parámetros son nombre y edad, declarados en la firma del método; los argumentos son "Ana" y 20, enviados en la llamada.

```
public void saludar(String nombre, int edad) { // parámetros
   System.out.println("Hola " + nombre + ", edad: " + edad);
}

saludar("Ana", 20); // argumentos
```

Ejercicio 4: comportamiento del paso por valor

El bloque de código mencionado en el enunciado tampoco aparece en la diapositiva, así que no se puede determinar con certeza qué instrucción modifica numero ni cuál es el valor final específico. En Java, para una variable primitiva, el método recibe una copia del valor: cambiar el parámetro dentro del método no cambia la variable original del main. Por ejemplo, si el método asigna 50 al parámetro, numero en main seguirá valiendo 10.

```
public static void cambiar(int valor) {
   valor = 50;
}

public static void main(String[] args) {
   int numero = 10;
   cambiar(numero);
   System.out.println(numero); // imprime 10 en este ejemplo
}
```

La salida anterior corresponde únicamente al ejemplo explicativo, no a un resultado confirmado del fragmento ausente de la diapositiva.

Ejercicio 5: método calcularPromedio

El método recibe tres valores double, calcula su media aritmética y devuelve el resultado como double.

```
public double calcularPromedio(double n1, double n2, double n3) {
    return (n1 + n2 + n3) / 3.0;
}
```

Ejercicio 6: validación de edad

El método devuelve boolean: false cuando la edad está fuera del intervalo permitido y true cuando es válida.

```
public boolean verificarEdad(int edad) {
    if (edad < 0 || edad > 120) {
        System.out.println("Edad inválida");
        return false;
    }
    return true;
}
```

Ejercicio 7: completar la clase Producto

La imagen o el esqueleto con espacios vacíos no se muestra en el contenido legible de la diapositiva. La solución siguiente se basa en los nombres de atributos y método dados explícitamente en el ejercicio 8: stock, precioUnitario y calcularValorTotal().

```
public class Producto {
    int stock;
    double precioUnitario;

    public double calcularValorTotal() {
        return stock * precioUnitario;
    }
}
```

Ejercicio 8: instanciación e invocación en main

Se crea el objeto p, se asignan los valores indicados, se invoca calcularValorTotal() y se imprime el resultado. Se asume que Producto tiene los atributos y método del ejercicio 7.

```
public class Principal {
    public static void main(String[] args) {
        Producto p = new Producto();
        p.stock = 10;
        p.precioUnitario = 25.5;

        double totalInventario = p.calcularValorTotal();
        System.out.println(totalInventario); // 255.0
    }
}
```

2.5 Resultados obtenidos

Se resolvieron los métodos de impresión, cálculo de descuento, bienvenida, promedio y validación; además, se mostró la invocación de métodos y la creación de un objeto Producto para calcular el valor del inventario. En los

ejercicios 1, 3, 4 y 7, algunos fragmentos de código no están visibles en la presentación consultada; por ello se identificaron explícitamente las soluciones ilustrativas o los supuestos utilizados, en lugar de atribuirlos al código original.

2.6 Habilidades blandas empleadas

- ☒ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☐ Responsabilidad

2.7 Conclusiones

1. El tipo de retorno debe reflejar el propósito del método: void para acciones sin resultado que devolver y un tipo de dato para resultados que el programa utilizará.
2. Los parámetros se declaran en la firma del método, mientras que los argumentos se proporcionan al invocarlo.
3. En los tipos primitivos, Java pasa una copia del valor al método; por eso una reasignación local no modifica la variable original.
4. Los ejercicios de inventario muestran cómo combinar atributos de un objeto y métodos con retorno para obtener un resultado reutilizable.

2.8 Recomendaciones

- Revisar que todo método con retorno tenga una instrucción return compatible.
- Verificar el código de las diapositivas 15, 17, 18 y 21 para contrastar los ejercicios ilustrativos con los fragmentos originales que no aparecen en el archivo legible.
- Probar los métodos con varios valores de entrada, incluidos los límites de validación.